04 Packet Capture & Anomaly Dashboard

04-pcap-anomaly-dashboard

Scapy PCAP analyser that windows traffic per source IP and flags port scans, traffic spikes and unknown protocols with transparent rules plus an Isolation Forest, scored against labelled synthetic captures.

Verified here: 25 passed; 0 failed; 0 skipped

Measured results

- Sample capture: 8,972 packets, 60 x 10 s windows, 996 (window, source) rows, parsed at ~7,276 packets/s
- Rules on 20 held-out seeds (standard): precision 225/225, recall 225/225
- Hard mode (low-and-slow scans, small bursts), 20 seeds: rules recall drops to 162/241 (67.2%); port-scan rule catches 46/113
- Isolation Forest in hard mode: recall 207/241 (85.9%) at precision 207/407 (50.9%), FP rate 1.01%
- Rules OR Forest in hard mode: recall 241/241, precision 241/441 (54.6%)
- Same seed -> byte-identical PCAP (SHA-256 checked in tests); 25/25 pytest tests pass

Data provenance and caveats

- All traffic is GENERATED by src/generate.py with Scapy from a fixed seed (no live capture, no public PCAP). Addresses are RFC 1918 / RFC 5737 documentation ranges; DNS names are RFC 2606 example domains.
- Ground-truth labels per (10 s window, source IP) are written by the same generator.
- Synthetic data only: results show the code works on its own labelled test traffic, not that it detects real attacks. Not production-validated.
- Rule thresholds were developed against seed 42 (the bundled sample), so sample numbers are optimistic; held-out seeds 1-20 and 101-120 were scored separately.
- The 100% rule scores in 'standard' mode reflect that the generated attacks are loud and well separated; 'hard' mode exists to show where fixed thresholds fail (rules recall 67.2%).
- Isolation Forest contamination is fixed at 0.02, so it always flags about 2% of rows; most of its false positives are servers (DNS resolver, web servers fanning out to many clients) and near-empty client windows.
- HTTP/HTTPS payloads are random bytes (handshake and sizes are realistic, content is not); only DNS is real protocol content.

Project contents

README.md, requirements.txt, cli.py, app.py, src/__init__.py, src/detect.py, src/evaluate.py, src/features.py, src/generate.py, src/parser.py, src/pipeline.py, src/report.py, tests/conftest.py, tests/helpers.py, tests/test_cli_app.py, tests/test_detect_eval.py, tests/test_generate.py, tests/test_parser_features.py, .gitignore, config/detector.json, data/sample.manifest.json, pytest.ini, scripts/run_demo.sh

Packet Capture & Anomaly Detection Dashboard

A Scapy-based PCAP analyser that cuts traffic into 10-second windows per source IP and flags **port scans**, **traffic spikes** and **unknown IP protocols** with transparent statistical rules plus an **Isolation Forest**, then scores itself honestly against labelled synthetic captures.

> **Data note:** every packet in this project is *generated* by `src/generate.py` from a fixed random seed. No live network is sniffed and no third-party capture is used. The precision/recall numbers below describe performance on that synthetic traffic only; they are **not** a production validation.

Problem

Network defenders often start from a packet capture and a question: *who is doing something unusual, and when?* Signature IDSs need known patterns; raw Wireshark browsing does not scale. This project shows the analyst workflow in code: parse a PCAP, turn packets into behavioural features per host and time window, apply explainable detection rules and an unsupervised model side by side, and measure each against ground truth with visible denominators.

How it works / Architecture

```
 generate (Scapy, seed)           analyze                              report /
dashboard
 ??????????????????????????   ???????????????????   ?????????????????????
??????????????????????????
 ? benign HTTP/HTTPS/DNS?   ? parser.py    ?   ? features.py      ?   ? detect.py
          ?
 ? + port scans         ????? PcapReader   ????? 10 s windows x   ????? rules: scan /
spike /?
 ? + traffic spikes     ?   ? per-packet   ?   ? src IP: packets, ?   ? unknown protocol
    ?
 ? + unknown IP protos  ?   ? DataFrame    ?   ? bytes, dst ports,?   ? IsolationForest on
   ?
 ? -> sample.pcap       ?   ???????????????????   ? SYN ratio, ...   ?   ?
RobustScaler(log1p)  ?
 ? -> labels.csv        ?                    ??????????????????????
??????????????????????????
 ??????????????????????????                                              ?
          ? ground truth per (window, src_ip)        evaluate.py  ??????????????
          ?????????????????????????????????????????????????????? TP/FP/FN/TN, precision, recall
                                              -> metrics.json, alerts.csv,
report.html, app.py
```

* **Generator** (`src/generate.py`): 14 internal clients make HTTP-like (TCP/80) and HTTPS-like (TCP/443) sessions with full 3-way handshake, request, response and FIN, plus real DNS A-record queries and answers (UDP/53) at a Poisson rate. The seed also places the anomalies on random hosts and windows:
  * `port_scan`: an external host (203.0.113.x) sends bare SYNs to hundreds of ports on 10.0.0.50;
  * `traffic_spike`: an internal client bursts hundreds of ~200-300 byte UDP datagrams;
  * `unknown_protocol`: an internal client emits IP protocol 253/254 (RFC 3692 experimental numbers).
  Every packet gets an explicit timestamp and IP ID, so the same seed gives a **byte-identical PCAP**. Labels are written for every (window, source IP) row in the capture.
* **Parser** (`src/parser.py`): streams the file with `PcapReader` (flat memory), keeps IPv4, extracts protocol, ports, length and TCP SYN/ACK flags; non-IP frames are counted

and skipped. `max_packets` bounds work on huge files.
* **Features** (`src/features.py`): windows are aligned to epoch multiples of 10 s, so a packet always lands in the same window no matter where the capture starts. Per (window, src_ip): packets, bytes, distinct destination IPs and ports, bare-SYN count and ratio, mean packet length, TCP/UDP/unknown-protocol counts.
* **Detectors** (`src/detect.py`, thresholds in `config/detector.json`):
  * port scan: at least 50 distinct destination ports **and** a bare-SYN ratio of at least 0.6;
  * traffic spike: robust z-score (median/MAD) of log(bytes) or log(packets) of at least 3.5, with at least 50 packets;
  * unknown protocol: any IP protocol outside {ICMP, TCP, UDP};
  * Isolation Forest (200 trees, contamination 0.02, fixed random_state) on `RobustScaler(log1p(features))`. It never sees labels.
* **Evaluation** (`src/evaluate.py`): one scored unit = one (window, src_ip) row; reports TP/FP/FN/TN, precision, recall, F1, false-positive rate and per-class hits, always with the raw fractions.

Security / design decisions

* **Synthetic, labelled data instead of a public PCAP.** Public captures rarely come with per-host, per-window ground truth, so precision/recall would be guesswork. Generating the traffic makes the labels exact and the experiment reproducible; the cost is realism, which is stated plainly.
* **Safe address space.** Only RFC 1918 (10.0.0.0/24) and RFC 5737 documentation ranges are used, and only RFC 2606 example domains. Nothing is sent on a network; packets go straight to a file.
* **Median/MAD instead of mean/std.** Attack windows are huge outliers. With mean/std the outliers inflate the baseline and hide themselves; the median and MAD barely move. Log transforms tame heavy-tailed byte counts.
* **Port-scan rule needs both breadth and bare SYNs.** "Many destination ports" alone would flag busy servers replying to many client ephemeral ports (they send SYN-ACK and data, not bare SYNs). A test covers this case.
* **Rules and ML side by side, not ML alone.** Rules are explainable ("436 ports, 100% SYN") and have near-zero false positives on this data, but only catch what their thresholds describe. The Isolation Forest finds unusual combinations without thresholds, at the cost of a fixed false-alarm budget. The union is the realistic SOC setup: high recall, with an analyst triaging.
* **Honest evaluation.** Thresholds were tuned while looking at seed 42, so the sample capture's scores are optimistic. `cli.py benchmark` generates fresh, unseen seeds (1-20 standard, 101-120 hard) and sums their confusion counts.
* **Bounded resources.** Streaming parser, optional packet cap, a 20 MB upload limit in the dashboard, and uploaded files only live in a temporary directory.

Project structure

```
04-pcap-anomaly-dashboard/
??? cli.py                  # generate | analyze | benchmark | report
??? app.py                  # Streamlit dashboard (4 tabs, optional PCAP upload)
??? config/detector.json    # rule thresholds + Isolation Forest parameters
??? src/
?   ??? generate.py         # deterministic labelled PCAP generator (Scapy)
?   ??? parser.py           # streaming PCAP -> per-packet DataFrame
?   ??? features.py         # 10 s window aggregation per source IP
?   ??? detect.py           # rules + Isolation Forest
?   ??? evaluate.py         # confusion counts, precision/recall with denominators
?   ??? pipeline.py         # analyze() and held-out benchmark()
?   ??? report.py           # matplotlib charts + self-contained HTML report
??? tests/                  # 25 pytest tests
??? scripts/run_demo.sh     # rebuilds every artifact in outputs/
```

??? data/sample.pcap   # generated, seed 42 (1.5 MB) + sample_labels.csv + manifest
??? outputs/   # analysis CSVs, metrics, benchmarks, charts, report.html,
test log
```

## Setup & how to run

```bash
pip install -r requirements.txt

python cli.py generate --seed 42 --out data/sample.pcap            # writes
data/sample_labels.csv too
python cli.py analyze  --pcap data/sample.pcap --labels data/sample_labels.csv --out
outputs/analysis
python cli.py benchmark --seeds 1-20 --difficulty standard --out
outputs/benchmark_standard.json
python cli.py benchmark --seeds 101-120 --difficulty hard --out
outputs/benchmark_hard.json
python cli.py report --bench outputs/benchmark_standard.json outputs/benchmark_hard.json

python -m pytest -v
streamlit run app.py
```

`bash scripts/run_demo.sh` runs the whole chain (about 2 minutes on a laptop CPU).
`analyze` also works on any real PCAP without `--labels` (it then reports alerts only, no
precision/recall). The dashboard accepts a PCAP upload as well.

Results (from the run in this repository)

Tests: 25 passed, 0 failed (`outputs/pytest_output.txt`, about 30 s). They cover byte-
identical generation for the same seed, parser fields (TCP flags, unknown protocol, non-IP
 frames, packet cap), exact window counts and alignment on hand-built captures, each rule
firing and *not* firing, Isolation Forest determinism, confusion-matrix maths, the CLI end
 to end, and the Streamlit app via AppTest.

Sample capture (seed 42, the development seed, so optimistic): 8,972 packets (1.5 MB),
 60 windows, 996 (window, source) rows, 10 of them anomalous. Parsed in 1.233 s (~7,276
packets/s with Scapy).

Detector	Precision	Recall	False positives
Rules	10/10 = 1.000	10/10 = 1.000	0
Isolation Forest	10/20 = 0.500	10/10 = 1.000	10
Rules OR Forest	10/20 = 0.500	10/10 = 1.000	10

Held-out seeds (never looked at while tuning):

| Benchmark | Rows (anomalous) | Rules P / R | Isolation Forest P / R | Rules OR Forest P
/ R |
|---|---|---|---|---|
| Standard, seeds 1-20 | 20,042 (225) | 225/225 / 225/225 | 214/411 / 214/225 | 225/422 /
225/225 |
| Hard, seeds 101-120 | 20,006 (241) | 162/162 / 162/241 (67.2%) | 207/407 (50.9%) /
207/241 (85.9%) | 241/441 (54.6%) / 241/241 |

Per class in hard mode: port-scan rule 46/113 vs forest 113/113; spike rule 76/88 vs
forest 88/88; unknown-protocol rule 40/40 vs forest 6/40.

What the numbers say:
```

* In standard mode the generated attacks are loud, so the rules are perfect. That says the synthetic task is easy, not that the rules are production-grade.
* "Hard" mode spreads scans over up to 3 windows with 20-150 ports (low and slow) and shrinks bursts. The fixed 50-port threshold then misses 67 of 113 scan windows, while the Isolation Forest still catches all of them, because a scanner's mix (many ports, all SYN, tiny packets) is unusual even below the threshold.
* The forest is weak exactly where the rule is strong: a handful of unknown-protocol packets look like a normal quiet host, so it caught only 6/40. The two detectors are complementary, and the union reached 241/241 recall in hard mode.
* The price is precision: contamination 0.02 makes the forest flag about 2% of rows every time. On the sample capture 8 of its 10 false positives were servers (5 windows of the DNS resolver 10.0.0.2 and 3 of web servers talking to many clients), and 2 were near-empty client windows (1-2 packets). Excluding known servers or adding per-host baselines would be the next step.

Charts (in `outputs/charts/`): `traffic_timeline.png` (packets per window by protocol, rule alerts marked), `feature_scatter.png` (packets vs distinct destination ports per row, coloured by ground truth, forest flags circled), `iforest_scores.png` (score distribution, benign vs anomalous), `benchmark_precision_recall.png` (held-out precision/recall per detector). `outputs/report.html` is a single-file report with the alert table and charts.

Sample output

```
$ python cli.py generate --seed 42 && python cli.py analyze --pcap data/sample.pcap
--labels data/sample_labels.csv
wrote data/sample.pcap (8972 packets, 1511276 bytes, sha256 31d779d3363caf48...)
wrote data/sample_labels.csv (996 labelled rows, 10 anomalous)
  scenario port_scan        src=203.0.113.66   windows 30..30 {'target': '10.0.0.50',
'ports': 436}
  scenario port_scan        src=203.0.113.67   windows 53..53 {'target': '10.0.0.50',
'ports': 425}
  scenario port_scan        src=203.0.113.68   windows 20..20 {'target': '10.0.0.50',
'ports': 422}
  scenario traffic_spike    src=10.0.0.20      windows 26..27 {'packets_per_window': 338,
 'payload': 273}
  scenario traffic_spike    src=10.0.0.11      windows 9..9 {'packets_per_window': 317,
'payload': 218}
  scenario traffic_spike    src=10.0.0.14      windows 48..49 {'packets_per_window': 275,
 'payload': 244}
  scenario unknown_protocol  src=10.0.0.17      windows 57..57 {'packets': 13}
  scenario unknown_protocol  src=10.0.0.15      windows 41..41 {'packets': 13}
parsed 8972 packets in 1.233 s (7276.9 pkt/s); 60 windows x sources = 996 rows
protocols seen: IP-253, IP-254, TCP, UDP
rule alerts: 10 {'traffic_spike': 5, 'port_scan': 3, 'unknown_protocol': 2}; isolation
forest flags: 20; combined: 20
evaluation vs labels (10 anomalous of 996 rows):
  rules            precision   10/10 = 1.000  recall   10/10 = 1.000  FP=0  FPR=0.0000
  isolation_forest  precision   10/20 = 0.500  recall   10/10 = 1.000  FP=10  FPR=0.0101
  combined         precision   10/20 = 0.500  recall   10/10 = 1.000  FP=10  FPR=0.0101
outputs in outputs/analysis/ (windows.csv, alerts.csv, window_totals.csv, metrics.json)

benchmark [standard] seeds 1-20: 225 anomalous of 20042 rows
  rules            precision   225/225 = 1.000  recall   225/225 = 1.000
  isolation_forest  precision   214/411 = 0.521  recall   214/225 = 0.951
  combined         precision   225/422 = 0.533  recall   225/225 = 1.000
  port_scan        rules 88/88  forest 88/88
  traffic_spike    rules 97/97  forest 97/97
  unknown_protocol  rules 40/40  forest 29/40
```

```
wrote outputs/benchmark_standard.json
benchmark [hard] seeds 101-120: 241 anomalous of 20006 rows
  rules             precision   162/162 = 1.000  recall   162/241 = 0.672
  isolation_forest  precision   207/407 = 0.509  recall   207/241 = 0.859
  combined          precision   241/441 = 0.546  recall   241/241 = 1.000
  port_scan         rules 46/113  forest 113/113
  traffic_spike     rules 76/88  forest 88/88
  unknown_protocol  rules 40/40  forest 6/40
wrote outputs/benchmark_hard.json
```

Limitations & future work

* **Synthetic only.** Real networks have NAT, bursty backups, CDNs and scanners that look nothing like this generator. None of these results transfers to a real network without testing on real labelled traffic (for example CIC-IDS2017 or UNSW-NB15).
* HTTP/HTTPS payloads are random bytes; only handshake, sizes and timing are realistic. DNS is real protocol content.
* Thresholds are global; production use needs per-host or per-role baselines (a DNS server should not be compared with a laptop).
* Isolation Forest is refit per capture and uses a fixed contamination, so it always raises about 2% of rows; a fitted-on-clean-baseline model with a calibrated score threshold would be better.
* IPv4 only; no flow reassembly or application-layer parsing; Scapy parses at a few thousand packets per second, so very large captures would need dpkt, pyshark or Zeek logs.
* No alert de-duplication across adjacent windows.

Resume bullets

* Built a Python/Scapy network-traffic anomaly detector that parses PCAPs, aggregates 10-second per-host windows and flags port scans, traffic spikes and unknown IP protocols through rule-based z-score detectors and an Isolation Forest, with a Streamlit dashboard and CLI.
* Designed a deterministic, labelled PCAP generator (byte-identical output per seed) and an evaluation harness that scores detectors on held-out seeds; rules reached 225/225 precision and recall on 20,042 standard-mode rows.
* Showed the limits of static thresholds: on low-and-slow "hard" traffic, rule recall fell to 67.2% (162/241), while combining rules with an Isolation Forest restored 241/241 recall at 54.6% precision (1.0% false-positive rate).
* Wrote 25 pytest tests covering deterministic generation, parsing, window aggregation, detector logic and the dashboard, all passing.

Packet Capture & Anomaly Dashboard — ACTUAL SAMPLE OUTPUT

```
$ bash scripts/run_demo.sh
wrote data/sample.pcap (8972 packets, 1511276 bytes, sha256 31d779d3363caf48...)
wrote data/sample_labels.csv (996 labelled rows, 10 anomalous)
  scenario port_scan         src=203.0.113.66   windows 30..30 {'target': '10.0.0.50', 'ports': 436}
  scenario port_scan         src=203.0.113.67   windows 53..53 {'target': '10.0.0.50', 'ports': 425}
  scenario port_scan         src=203.0.113.68   windows 20..20 {'target': '10.0.0.50', 'ports': 422}
  scenario traffic_spike     src=10.0.0.20      windows 26..27 {'packets_per_window': 338, 'payload': 273}
  scenario traffic_spike     src=10.0.0.11      windows 9..9 {'packets_per_window': 317, 'payload': 218}
  scenario traffic_spike     src=10.0.0.14      windows 48..49 {'packets_per_window': 275, 'payload': 244}
  scenario unknown_protocol  src=10.0.0.17      windows 57..57 {'packets': 13}
  scenario unknown_protocol  src=10.0.0.15      windows 41..41 {'packets': 13}
parsed 8972 packets in 1.233 s (7276.9 pkt/s); 60 windows x sources = 996 rows
protocols seen: IP-253, IP-254, TCP, UDP
rule alerts: 10 {'traffic_spike': 5, 'port_scan': 3, 'unknown_protocol': 2}; isolation forest flags: 20; combined: 20
evaluation vs labels (10 anomalous of 996 rows):
  rules             precision   10/10 = 1.000  recall   10/10 = 1.000  FP=0  FPR=0.0000
  isolation_forest  precision   10/20 = 0.500  recall   10/10 = 1.000  FP=10  FPR=0.0101
  combined          precision   10/20 = 0.500  recall   10/10 = 1.000  FP=10  FPR=0.0101
outputs in outputs/analysis/ (windows.csv, alerts.csv, window_totals.csv, metrics.json)
benchmark [standard] seeds 1-20: 225 anomalous of 20042 rows
  rules             precision   225/225 = 1.000  recall   225/225 = 1.000
  isolation_forest  precision   214/411 = 0.521  recall   214/225 = 0.951
  combined          precision   225/422 = 0.533  recall   225/225 = 1.000
  port_scan         rules 88/88  forest 88/88
  traffic_spike     rules 97/97  forest 97/97
  unknown_protocol  rules 40/40  forest 29/40
wrote outputs/benchmark_standard.json
benchmark [hard] seeds 101-120: 241 anomalous of 20006 rows
  rules             precision   162/162 = 1.000  recall   162/241 = 0.672
  isolation_forest  precision   207/407 = 0.509  recall   207/241 = 0.859
  combined          precision   241/441 = 0.546  recall   241/241 = 1.000
  port_scan         rules 46/113  forest 113/113
  traffic_spike     rules 76/88  forest 88/88
  unknown_protocol  rules 40/40  forest 6/40
wrote outputs/benchmark_hard.json
wrote outputs/report.html
wrote outputs/charts/traffic_timeline.png
wrote outputs/charts/feature_scatter.png
wrote outputs/charts/iforest_scores.png
wrote outputs/charts/benchmark_precision_recall.png

```

04-pcap-anomaly-dashboard/requirements.txt

```
scapy>=2.5
numpy>=1.24
pandas>=2.0
scikit-learn>=1.3
matplotlib>=3.7
plotly>=5.18
streamlit>=1.40
pytest>=7.4
pytest-timeout>=2.2

```

04-pcap-anomaly-dashboard/cli.py

```
#!/usr/bin/env python
"""PCAP anomaly detection CLI.

  python cli.py generate  --seed 42 --out data/sample.pcap
  python cli.py analyze   --pcap data/sample.pcap --labels data/sample_labels.csv
  python cli.py benchmark --seeds 1-10 --difficulty standard
  python cli.py report    --analysis outputs/analysis
"""
from __future__ import annotations

import argparse
import json
import logging
import sys
from pathlib import Path

logging.getLogger("scapy.runtime").setLevel(logging.ERROR)


def _seeds(text: str) -> list[int]:
    out: list[int] = []
    for part in text.split(","):
        if "-" in part:
            a, b = part.split("-"); out.extend(range(int(a), int(b) + 1))
        else:
            out.append(int(part))
    return out


def cmd_generate(a) -> int:
    from src.generate import GenConfig, generate
    labels = a.labels or str(Path(a.out).with_name(Path(a.out).stem + "_labels.csv"))
    m = generate(GenConfig(seed=a.seed, duration_s=a.duration, difficulty=a.difficulty), a.out, la
bels)
    print(f"wrote {a.out} ({m['packets']} packets, {m['pcap_bytes']} bytes, sha256 {m['pcap_sha256
'][:16]}...)")
    print(f"wrote {labels} ({m['labelled_rows']} labelled rows, {m['anomalous_rows']} anomalous)")
    for s in m["scenarios"]:
        print(f"  scenario {s['kind']:<17} src={s['src_ip']:<14} windows {s['start_window']}..{s['
start_window'] + s['n_windows'] - 1} {s['detail']}")
    return 0


def cmd_analyze(a) -> int:
    from src.pipeline import analyze
    s = analyze(a.pcap, a.out, a.labels, a.config)
    print(f"parsed {s['packets_parsed']} packets in {s['parse_seconds']} s ({s['packets_per_second
']} pkt/s); "
          f"{s['windows']} windows x sources = {s['window_rows']} rows")
    print(f"protocols seen: {', '.join(s['protocols'])}")
    print(f"rule alerts: {s['rule_alerts']} {s['rule_alerts_by_type']}; isolation forest flags: {s
['if_alerts']}; "
          f"combined: {s['combined_alerts']}")
    ev = s.get("evaluation")
    if ev:
        print(f"evaluation vs labels ({ev['anomalous_rows']} anomalous of {ev['rows']} rows):")
        for k in ["rules", "isolation_forest", "combined"]:
            e = ev[k]
            print(f"  {k:<17} precision {e['precision_str']:>7} = {e['precision']:.3f}  "
                  f"recall {e['recall_str']:>7} = {e['recall']:.3f}  FP={e['fp']}  FPR={e['false_p
ositive_rate']:.4f}")
    print(f"outputs in {a.out}/ (windows.csv, alerts.csv, window_totals.csv, metrics.json)")
    return 0


```

```
60 def cmd_benchmark(a) -> int:
61     from src.pipeline import benchmark
62     b = benchmark(_seeds(a.seeds), a.work, a.config, a.difficulty)
63     Path(a.out).parent.mkdir(parents=True, exist_ok=True)
64     Path(a.out).write_text(json.dumps(b, indent=2))
65     print(f"benchmark [{a.difficulty}] seeds {a.seeds}: {b['anomalous_rows']} anomalous of {b['row
   s']} rows")
66     for k in ["rules", "isolation_forest", "combined"]:
67         e = b[k]
68         print(f"  {k:<17} precision {e['precision_str']:>9} = {e['precision']:.3f}  recall {e['rec
   all_str']:>9} = {e['recall']:.3f}")
69     for cls, c in b["per_class"].items():
70         print(f"  {cls:<17} rules {c['rules_detected']}/{c['rows']}  forest {c['if_detected']}/{c[
   'rows']}")
71     print(f"wrote {a.out}")
72     return 0
73
74
75 def cmd_report(a) -> int:
76     from src.report import build_report
77     bench = {}
78     for p in a.bench or []:
79         d = json.loads(Path(p).read_text()); bench[f"{d['difficulty']} ({len(d['seeds'])} seeds)"]
    = d
80     made = build_report(a.analysis, a.html, a.charts, bench or None)
81     print(f"wrote {a.html}"); [print(f"wrote {p}") for p in made]
82     return 0
83
84
85 def main(argv=None) -> int:
86     p = argparse.ArgumentParser(description="Synthetic PCAP anomaly detection (rules + Isolation F
   orest)")
87     sub = p.add_subparsers(dest="cmd", required=True)
88     g = sub.add_parser("generate", help="write a deterministic labelled synthetic PCAP")
89     g.add_argument("--seed", type=int, default=42); g.add_argument("--duration", type=int, default
   =600)
90     g.add_argument("--difficulty", choices=["standard", "hard"], default="standard")
91     g.add_argument("--out", default="data/sample.pcap"); g.add_argument("--labels")
92     g.set_defaults(fn=cmd_generate)
93     an = sub.add_parser("analyze", help="parse a PCAP, aggregate windows, detect anomalies")
94     an.add_argument("--pcap", required=True); an.add_argument("--labels", help="optional ground-tr
   uth CSV")
95     an.add_argument("--out", default="outputs/analysis"); an.add_argument("--config")
96     an.set_defaults(fn=cmd_analyze)
97     b = sub.add_parser("benchmark", help="score detectors on freshly generated held-out seeds")
98     b.add_argument("--seeds", default="1-10"); b.add_argument("--difficulty", choices=["standard",
    "hard"], default="standard")
99     b.add_argument("--work", default="data/bench_tmp"); b.add_argument("--config")
100    b.add_argument("--out", default="outputs/benchmark.json"); b.set_defaults(fn=cmd_benchmark)
101    r = sub.add_parser("report", help="charts + self-contained HTML report")
102    r.add_argument("--analysis", default="outputs/analysis"); r.add_argument("--html", default="ou
   tputs/report.html")
103    r.add_argument("--charts", default="outputs/charts"); r.add_argument("--bench", nargs="*")
104    r.set_defaults(fn=cmd_report)
105    a = p.parse_args(argv)
106    return a.fn(a)
107
108
109 if __name__ == "__main__":
110     sys.exit(main())
111
```

```python
"""Streamlit dashboard: PCAP anomaly detection on synthetic, labelled traffic.

Run:  streamlit run app.py
Reads outputs/analysis/*.csv (produced by `python cli.py analyze ...`). If
they are missing it regenerates the deterministic sample and analyses it, so
the app works offline from a fresh clone.
"""
from __future__ import annotations

import json
import tempfile
from pathlib import Path

import pandas as pd
import plotly.express as px
import streamlit as st

ROOT = Path(__file__).resolve().parent
ANALYSIS = ROOT / "outputs" / "analysis"
SAMPLE_PCAP, SAMPLE_LABELS = ROOT / "data" / "sample.pcap", ROOT / "data" / "sample_labels.csv"
MAX_UPLOAD_MB = 20

st.set_page_config(page_title="PCAP Anomaly Dashboard", layout="wide")


@st.cache_data(show_spinner=False)
def load_default():
    if not (ANALYSIS / "windows.csv").exists():
        from src.generate import GenConfig, generate
        from src.pipeline import analyze
        if not SAMPLE_PCAP.exists():
            generate(GenConfig(seed=42), SAMPLE_PCAP, SAMPLE_LABELS)
        analyze(SAMPLE_PCAP, ANALYSIS, SAMPLE_LABELS)
    return (pd.read_csv(ANALYSIS / "windows.csv"), pd.read_csv(ANALYSIS / "window_totals.csv"),
            json.loads((ANALYSIS / "metrics.json").read_text()))


def analyze_upload(data: bytes):
    from src.pipeline import analyze
    with tempfile.TemporaryDirectory() as td:
        p = Path(td) / "upload.pcap"; p.write_bytes(data)
        analyze(p, Path(td) / "out")
        o = Path(td) / "out"
        return (pd.read_csv(o / "windows.csv"), pd.read_csv(o / "window_totals.csv"),
                json.loads((o / "metrics.json").read_text()))


st.title("Packet Capture Anomaly Detection")
st.caption("Synthetic, labelled traffic generated with Scapy (seed 42). Rule-based detectors + Isolation Forest. "
           "Results are measured on generated data only; they are not production-validated.")

with st.sidebar:
    st.header("Data")
    up = st.file_uploader(f"Analyse your own PCAP (max {MAX_UPLOAD_MB} MB, stays in memory/temp)", type=["pcap", "pcapng", "cap"])
    if up is not None and up.size > MAX_UPLOAD_MB * 1024 * 1024:
        st.error("File too large."); up = None

if up is not None:
    windows, totals, metrics = analyze_upload(up.getvalue())
    st.info(f"Showing uploaded file {up.name} (no ground-truth labels, so no precision/recall).")
else:
    windows, totals, metrics = load_default()

```

```
with st.sidebar:
    st.header("Filters")
    srcs = sorted(windows["src_ip"].unique())
    pick = st.multiselect("Source IPs", srcs, default=[])
    only_flagged = st.checkbox("Only flagged rows in table", value=True)
view = windows[windows["src_ip"].isin(pick)] if pick else windows

c = st.columns(5)
c[0].metric("Packets", f"{metrics['packets_parsed']:,}")
c[1].metric("Windows (10 s)", metrics["windows"])
c[2].metric("Source IPs", metrics["unique_src_ips"])
c[3].metric("Rule alerts", metrics["rule_alerts"])
c[4].metric("Isolation Forest flags", metrics["if_alerts"])

tab_tl, tab_anom, tab_ml, tab_eval = st.tabs(["Traffic timeline", "Anomaly table", "Feature space", "Evaluation"])

with tab_tl:
    fig = px.bar(totals, x="window_start", y="packets", color="proto_name",
                 labels={"window_start": "Window start (s)", "packets": "Packets", "proto_name": "Protocol"},
                 title="Packets per 10-second window by protocol")
    for _, r in view[view["rule_flag"]].iterrows():
        fig.add_vline(x=r["window_start"], line_dash="dash", line_color="red", opacity=.5)
    st.plotly_chart(fig, width="stretch")
    per_src = view.groupby("window_start", as_index=False)["bytes"].sum()
    st.plotly_chart(px.line(per_src, x="window_start", y="bytes", markers=True,
                            title="Bytes per window (selected sources)"), width="stretch")

with tab_anom:
    tbl = view[view["combined_flag"]] if only_flagged else view
    cols = [c for c in ["window_start", "src_ip", "packets", "bytes", "unique_dst_ips", "unique_dst_ports",
                        "syn_ratio", "unknown_proto_packets", "z_bytes", "rule_reason", "if_score", "if_flag", "label"]
            if c in tbl]
    st.write(f"{len(tbl)} rows")
    st.dataframe(tbl[cols].sort_values("if_score", ascending=False), width="stretch", hide_index=True)
    st.download_button("Download alerts CSV", tbl[cols].to_csv(index=False), "alerts.csv", "text/csv")

with tab_ml:
    color = "label" if "label" in view else "rule_reason"
    fig = px.scatter(view.assign(ports=view["unique_dst_ports"] + 1), x="packets", y="ports", color=color,
                     symbol="if_flag", log_x=True, log_y=True, hover_data=["src_ip", "window_start", "if_score"],
                     labels={"ports": "Distinct dst ports + 1"}, title="Window features (symbol = Isolation Forest flag)")
    st.plotly_chart(fig, width="stretch")
    st.plotly_chart(px.histogram(view, x="if_score", color=color, nbins=50, log_y=True,
                                 title="Isolation Forest anomaly score"), width="stretch")

with tab_eval:
    ev = metrics.get("evaluation")
    if not ev:
        st.write("No labels for this capture.")
    else:
        st.subheader(f"Sample capture: {ev['anomalous_rows']} anomalous of {ev['rows']} (window, source) rows")
        rows = [{"detector": n, "precision": f"{ev[k]['precision_str']} = {ev[k]['precision']:.3f}",
                 "recall": f"{ev[k]['recall_str']} = {ev[k]['recall']:.3f}", "false positives": ev[k]["fp"],
```

```
                 "FP rate": ev[k]["false_positive_rate"]}
                for n, k in [("Rules", "rules"), ("Isolation Forest", "isolation_forest"), ("Rules OR Forest", "combined")]]
        st.table(pd.DataFrame(rows))
        st.caption("Thresholds were developed on seed 42, so these sample numbers are optimistic. See held-out seeds below.")
    for name in ["standard", "hard"]:
        p = ROOT / "outputs" / f"benchmark_{name}.json"
        if p.exists():
            b = json.loads(p.read_text())
            st.subheader(f"Held-out seeds ({name}): {b['anomalous_rows']} anomalous of {b['rows']} rows")
            st.table(pd.DataFrame([{"detector": k, "precision": f"{b[k]['precision_str']} = {b[k]['precision']:.3f}",
                                    "recall": f"{b[k]['recall_str']} = {b[k]['recall']:.3f}"}
                                   for k in ["rules", "isolation_forest", "combined"]]))
            st.table(pd.DataFrame(b["per_class"]).T)

```

04-pcap-anomaly-dashboard/src/__init__.py

```
"""PCAP anomaly detection toolkit (synthetic, labelled, offline)."""
__version__ = "1.0.0"

```

```
"""Two independent detectors over window features.

1. Rules (transparent, explainable):
   * port_scan        many distinct destination ports + mostly bare SYNs
   * traffic_spike    robust z-score of log(bytes) and log(packets) above a
                      threshold (median/MAD, so the outliers themselves do not
                      inflate the baseline the way mean/std would)
   * unknown_protocol any IP protocol outside the known list
2. Isolation Forest (unsupervised ML) on RobustScaler-scaled log features.
   It never sees the labels; ``contamination`` sets the share of rows it flags.

Each window row ends with ``rule_flag``, ``rule_reason``, ``if_score``,
``if_flag`` and ``combined_flag`` (rules OR forest).
"""
from __future__ import annotations

import json
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import RobustScaler

DEFAULT_CONFIG = Path(__file__).resolve().parent.parent / "config" / "detector.json"
ML_FEATURES = ["packets", "bytes", "unique_dst_ips", "unique_dst_ports",
               "syn_ratio", "mean_pkt_len", "unknown_proto_packets"]


def load_config(path: str | Path | None = None) -> dict:
    return json.loads(Path(path or DEFAULT_CONFIG).read_text())


def robust_z(x: pd.Series) -> pd.Series:
    """(x - median) / (1.4826 * MAD); MAD of 0 falls back to std, then 1."""
    med = x.median()
    mad = (x - med).abs().median() * 1.4826
    if not mad or np.isnan(mad):
        mad = x.std() or 1.0
    return (x - med) / mad


def rule_detect(feat: pd.DataFrame, cfg: dict) -> pd.DataFrame:
    df = feat.copy()
    ps, sp, up = cfg["port_scan"], cfg["traffic_spike"], cfg["unknown_protocol"]
    df["z_bytes"] = robust_z(np.log1p(df["bytes"])).round(3)
    df["z_packets"] = robust_z(np.log1p(df["packets"])).round(3)

    scan = (df["unique_dst_ports"] >= ps["min_unique_dst_ports"]) & (df["syn_ratio"] >= ps["min_sy
n_ratio"])
    unknown = df["unknown_proto_packets"] >= up["min_packets"]
    spike = ((df[["z_bytes", "z_packets"]].max(axis=1) >= sp["robust_z_threshold"])
             & (df["packets"] >= sp["min_packets"]) & ~scan)

    # Priority when several fire: scan > unknown protocol > spike.
    reason = np.select([scan, unknown, spike], ["port_scan", "unknown_protocol", "traffic_spike"],
 default="")
    df["rule_reason"] = reason
    df["rule_flag"] = df["rule_reason"] != ""
    return df


def isolation_forest(feat: pd.DataFrame, cfg: dict) -> pd.DataFrame:
    df = feat.copy()
    p = cfg["isolation_forest"]
```

```
    if len(df) < 2:
        df["if_score"], df["if_flag"] = 0.0, False
        return df
    X = np.log1p(df[ML_FEATURES].astype(float).clip(lower=0))
    Xs = RobustScaler().fit_transform(X)
    model = IsolationForest(n_estimators=p["n_estimators"], contamination=p["contamination"],
                            random_state=p["random_state"])
    model.fit(Xs)
    # score_samples: lower = more anomalous; negate so higher = more anomalous.
    df["if_score"] = (-model.score_samples(Xs)).round(4)
    df["if_flag"] = model.predict(Xs) == -1
    return df


def detect(feat: pd.DataFrame, cfg: dict | None = None) -> pd.DataFrame:
    cfg = cfg or load_config()
    df = rule_detect(feat, cfg)
    df = isolation_forest(df, cfg)
    df["combined_flag"] = df["rule_flag"] | df["if_flag"]
    return df

```

04-pcap-anomaly-dashboard/src/evaluate.py

```
"""Compare detector output with generator ground truth.

The scored unit is the (window_start, src_ip) row. Every metric is reported
with its raw counts (TP/FP/FN/TN) so the denominators are visible.
"""
from __future__ import annotations

import pandas as pd


def confusion(y_true, y_pred) -> dict:
    y_true = pd.Series(y_true).astype(bool).to_numpy()
    y_pred = pd.Series(y_pred).astype(bool).to_numpy()
    tp = int((y_true & y_pred).sum()); fp = int((~y_true & y_pred).sum())
    fn = int((y_true & ~y_pred).sum()); tn = int((~y_true & ~y_pred).sum())
    prec = tp / (tp + fp) if tp + fp else 0.0
    rec = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * prec * rec / (prec + rec) if prec + rec else 0.0
    fpr = fp / (fp + tn) if fp + tn else 0.0
    return {"tp": tp, "fp": fp, "fn": fn, "tn": tn, "precision": round(prec, 4),
            "recall": round(rec, 4), "f1": round(f1, 4), "false_positive_rate": round(fpr, 5),
            "precision_str": f"{tp}/{tp + fp}", "recall_str": f"{tp}/{tp + fn}"}


def join_labels(det: pd.DataFrame, labels: pd.DataFrame) -> pd.DataFrame:
    """Left-join labels; rows missing from the label file count as benign."""
    m = det.merge(labels, on=["window_start", "src_ip"], how="left")
    m["label"] = m["label"].fillna("benign")
    m["is_anomaly"] = m["label"] != "benign"
    return m


def evaluate(det: pd.DataFrame, labels: pd.DataFrame) -> dict:
    m = join_labels(det, labels)
    res = {"rows": int(len(m)), "anomalous_rows": int(m["is_anomaly"].sum()),
           "rules": confusion(m["is_anomaly"], m["rule_flag"]),
           "isolation_forest": confusion(m["is_anomaly"], m["if_flag"]),
           "combined": confusion(m["is_anomaly"], m["combined_flag"]), "per_class": {}}
    for cls in sorted(set(m["label"]) - {"benign"}):
        sub = m[m["label"] == cls]
        res["per_class"][cls] = {
            "rows": int(len(sub)),
            "rules_detected": int(sub["rule_flag"].sum()),
            "rules_correct_type": int((sub["rule_reason"] == cls).sum()),
            "if_detected": int(sub["if_flag"].sum()),
        }
    return res

```

```
"""Aggregate packets into fixed time windows per source IP.

One row = one (window_start, src_ip) pair. window_start is seconds since the
capture's first window boundary (the first packet's epoch time floored to a
multiple of the window size, default 10 s).
"""
from __future__ import annotations

import pandas as pd

KNOWN_PROTOS = (1, 6, 17)

FEATURES = ["packets", "bytes", "unique_dst_ips", "unique_dst_ports",
            "syn_packets", "syn_ratio", "mean_pkt_len", "unknown_proto_packets",
            "tcp_packets", "udp_packets"]


def assign_windows(pkts: pd.DataFrame, window_s: int = 10, t0: float | None = None) -> pd.DataFram
e:
    """Add a ``window_start`` column (int seconds from capture start)."""
    df = pkts.copy()
    if df.empty:
        df["window_start"] = pd.Series(dtype="int64")
        return df
    if t0 is None:
        # Align windows to wall-clock multiples of window_s (e.g. :00, :10, :20),
        # so the same packet always lands in the same window regardless of
        # where the capture happened to start.
        t0 = (df["ts"].min() // window_s) * window_s
    df["window_start"] = (((df["ts"] - t0) // window_s) * window_s).astype("int64")
    return df


def window_features(pkts: pd.DataFrame, window_s: int = 10,
                    known_protos=KNOWN_PROTOS, t0: float | None = None) -> pd.DataFrame:
    """Per-(window, src_ip) behavioural features used by both detectors."""
    df = assign_windows(pkts, window_s, t0)
    if df.empty:
        return pd.DataFrame(columns=["window_start", "src_ip", *FEATURES])
    df["is_syn_only"] = df["syn"] & ~df["ack"]
    df["is_unknown"] = ~df["proto"].isin(known_protos)
    df["is_tcp"] = df["proto"] == 6
    df["is_udp"] = df["proto"] == 17
    df["dport_valid"] = df["dport"].where(df["dport"] >= 0)
    g = df.groupby(["window_start", "src_ip"], sort=True)
    out = g.agg(
        packets=("length", "size"),
        bytes=("length", "sum"),
        unique_dst_ips=("dst_ip", "nunique"),
        unique_dst_ports=("dport_valid", "nunique"),
        syn_packets=("is_syn_only", "sum"),
        unknown_proto_packets=("is_unknown", "sum"),
        tcp_packets=("is_tcp", "sum"),
        udp_packets=("is_udp", "sum"),
        mean_pkt_len=("length", "mean"),
    ).reset_index()
    out["syn_ratio"] = (out["syn_packets"] / out["packets"]).round(4)
    out["mean_pkt_len"] = out["mean_pkt_len"].round(2)
    for c in ["packets", "bytes", "unique_dst_ips", "unique_dst_ports", "syn_packets",
              "unknown_proto_packets", "tcp_packets", "udp_packets"]:
        out[c] = out[c].astype("int64")
    return out[["window_start", "src_ip", *FEATURES]]


def window_totals(pkts: pd.DataFrame, window_s: int = 10, t0: float | None = None) -> pd.DataFrame
```

```
:
    """Whole-capture traffic per window and protocol (for timeline charts)."""
    df = assign_windows(pkts, window_s, t0)
    return (df.groupby(["window_start", "proto_name"]).agg(packets=("length", "size"), bytes=("length", "sum"))
              .reset_index())

```

```
"""Deterministic, labelled synthetic PCAP generator.

Nothing here sniffs a real network. Every packet is built with Scapy from a
seeded NumPy random generator and given an explicit timestamp, so the same
seed always produces a byte-identical PCAP file.

Address plan (all non-routable / documentation ranges):
  * 10.0.0.0/24        internal clients and servers (RFC 1918)
  * 198.51.100.0/24    "internet" web servers (RFC 5737 TEST-NET-2)
  * 203.0.113.0/24     external scanners (RFC 5737 TEST-NET-3)

Scenarios
  benign            HTTP-like (TCP/80) and HTTPS-like (TCP/443) sessions with a
                    full handshake + random payload bytes, and real DNS A-record
                    queries/answers (UDP/53, RFC 2606 example domains) from
                    internal clients at a Poisson rate.
  port_scan         one external host SYN-probes hundreds of TCP ports on one
                    internal server within one or two windows.
  traffic_spike     one internal client suddenly bursts large UDP datagrams
                    (exfiltration / flood shaped) for one or two windows.
  unknown_protocol  one internal client emits IP protocol numbers 253/254
                    (RFC 3692 "experimental") that the analyser does not know.

Ground truth is written as one label per (window_start, src_ip) — the same
unit the detector scores — so evaluation needs no fuzzy matching.
"""
from __future__ import annotations

import csv
import hashlib
import json
from dataclasses import dataclass, field, asdict
from pathlib import Path

import numpy as np
from scapy.all import IP, TCP, UDP, Ether, Raw, wrpcap  # type: ignore
from scapy.layers.dns import DNS, DNSQR, DNSRR  # type: ignore

BASE_EPOCH = 1_700_000_000.0          # fixed capture start (2023-11-14 UTC)
WINDOW_SECONDS = 10
DNS_SERVER = "10.0.0.2"
WEB_SERVERS = [f"198.51.100.{i}" for i in (10, 20, 30, 40)]
INTERNAL_SERVER = "10.0.0.50"
DOMAINS = ["www.example.com", "mail.example.org", "cdn.example.net", "api.example.com",
           "docs.example.org", "updates.example.net"]          # RFC 2606 names
MAC_A = "02:00:00:00:00:01"           # locally administered MACs
MAC_B = "02:00:00:00:00:02"


@dataclass
class GenConfig:
    seed: int = 42
    duration_s: int = 600                 # 60 windows of 10 s
    n_clients: int = 14
    benign_rate_per_s: float = 0.18       # flows per client per second
    n_port_scans: int = 3
    n_spikes: int = 3
    n_unknown: int = 2
    # "standard": attacks are loud and well separated from benign traffic.
    # "hard": low-and-slow scans, small bursts and few unknown-protocol packets,
    #         to show where fixed thresholds stop working.
    difficulty: str = "standard"


@dataclass
```

```
class Scenario:
    kind: str
    src_ip: str
    start_window: int
    n_windows: int
    detail: dict = field(default_factory=dict)


def _payload(rng: np.random.Generator, n: int) -> bytes:
    """Deterministic pseudo-random payload bytes (not real content)."""
    return rng.integers(0, 256, size=n, dtype=np.uint8).tobytes()


class _Builder:
    """Collects packets with explicit timestamps."""

    def __init__(self, rng: np.random.Generator):
        self.rng = rng
        self.packets = []
        self._ip_id = 0

    def _ip(self, src: str, dst: str, proto: int | None = None) -> IP:
        self._ip_id = (self._ip_id + 1) % 65536
        kw = dict(src=src, dst=dst, id=self._ip_id, ttl=64)
        if proto is not None:
            kw["proto"] = proto
        return IP(**kw)

    def add(self, pkt, t: float) -> None:
        pkt.time = round(BASE_EPOCH + t, 6)
        self.packets.append(pkt)

    # ---- benign building blocks --------------------------------------------
    def tcp_session(self, client: str, server: str, dport: int, t: float,
                    req: int, resp: int) -> None:
        r = self.rng
        sport = int(r.integers(32768, 61000))
        seq_c, seq_s = int(r.integers(1, 2**31)), int(r.integers(1, 2**31))
        e = Ether(src=MAC_A, dst=MAC_B)
        rtt = float(r.uniform(0.01, 0.08))
        self.add(e / self._ip(client, server) / TCP(sport=sport, dport=dport, flags="S", seq=seq_c
), t)
        self.add(e / self._ip(server, client) / TCP(sport=dport, dport=sport, flags="SA", seq=seq_
s, ack=seq_c + 1), t + rtt / 2)
        self.add(e / self._ip(client, server) / TCP(sport=sport, dport=dport, flags="A", seq=seq_c
 + 1, ack=seq_s + 1), t + rtt)
        self.add(e / self._ip(client, server) / TCP(sport=sport, dport=dport, flags="PA", seq=seq_
c + 1, ack=seq_s + 1) / Raw(_payload(r, req)), t + rtt + 0.001)
        self.add(e / self._ip(server, client) / TCP(sport=dport, dport=sport, flags="PA", seq=seq_
s + 1, ack=seq_c + 1 + req) / Raw(_payload(r, resp)), t + 1.5 * rtt)
        self.add(e / self._ip(client, server) / TCP(sport=sport, dport=dport, flags="FA", seq=seq_
c + 1 + req, ack=seq_s + 1 + resp), t + 2 * rtt)

    def dns_exchange(self, client: str, t: float) -> None:
        r = self.rng
        sport = int(r.integers(32768, 61000))
        e = Ether(src=MAC_A, dst=MAC_B)
        qid = int(r.integers(0, 65536))
        name = str(r.choice(DOMAINS))
        answer = f"192.0.2.{int(r.integers(1, 255))}"            # RFC 5737 TEST-NET-1
        self.add(e / self._ip(client, DNS_SERVER) / UDP(sport=sport, dport=53) /
                 DNS(id=qid, rd=1, qd=DNSQR(qname=name)), t)
        self.add(e / self._ip(DNS_SERVER, client) / UDP(sport=53, dport=sport) /
                 DNS(id=qid, qr=1, rd=1, ra=1, qd=DNSQR(qname=name),
                     an=DNSRR(rrname=name, type="A", ttl=300, rdata=answer)),
```

```
                t + float(r.uniform(0.002, 0.03)))


def _plan_scenarios(cfg: GenConfig, rng: np.random.Generator, clients: list[str]) -> list[Scenario
]:
    """Choose non-overlapping windows / hosts for each anomaly from the seed."""
    n_windows = cfg.duration_s // WINDOW_SECONDS
    free = list(range(2, n_windows - 2))          # keep first/last windows clean
    rng.shuffle(free)
    used: set[int] = set()
    out: list[Scenario] = []

    def take(span: int) -> int:
        # Prefer windows that keep a gap around every scenario (cleaner charts);
        # short captures fall back to overlapping windows. Labels stay exact
        # either way because each scenario uses its own source IP.
        for need_gap in (True, False):
            for w0 in list(free):
                ws = set(range(w0, w0 + span))
                halo = (ws | {w0 - 1, w0 + span}) if need_gap else ws
                if ws <= set(range(n_windows)) and not halo & used:
                    used.update(ws)
                    free.remove(w0)
                    return w0
        if free:
            w0 = free.pop(0)
            return w0 if w0 + span <= n_windows else n_windows - span
        raise ValueError("capture too short for the requested scenarios")

    if cfg.difficulty not in ("standard", "hard"):
        raise ValueError("difficulty must be 'standard' or 'hard'")
    hard = cfg.difficulty == "hard"
    scan_ports = (20, 150) if hard else (120, 500)
    scan_span = (1, 4) if hard else (1, 3)
    spike_pkts = (30, 150) if hard else (120, 400)
    unknown_pkts = (1, 4) if hard else (6, 20)

    spare_clients = list(clients)
    rng.shuffle(spare_clients)
    for i in range(cfg.n_port_scans):
        span = int(rng.integers(*scan_span))
        out.append(Scenario("port_scan", f"203.0.113.{66 + i}", take(span), span,
                            {"target": INTERNAL_SERVER, "ports": int(rng.integers(*scan_ports))}))
    for _ in range(cfg.n_spikes):
        span = int(rng.integers(1, 3))
        out.append(Scenario("traffic_spike", spare_clients.pop(), take(span), span,
                            {"packets_per_window": int(rng.integers(*spike_pkts)),
                             "payload": int(rng.integers(180, 320))}))
    for _ in range(cfg.n_unknown):
        out.append(Scenario("unknown_protocol", spare_clients.pop(), take(1), 1,
                            {"packets": int(rng.integers(*unknown_pkts))}))
    return out


def generate(cfg: GenConfig, pcap_path: str | Path, labels_path: str | Path) -> dict:
    """Write a PCAP and its ground-truth labels; return a manifest dict."""
    rng = np.random.default_rng(cfg.seed)
    clients = [f"10.0.0.{10 + i}" for i in range(cfg.n_clients)]
    b = _Builder(rng)
    n_windows = cfg.duration_s // WINDOW_SECONDS
    scenarios = _plan_scenarios(cfg, rng, clients)

    # ---- benign background ------------------------------------------------
    for c in clients:
        n_flows = int(rng.poisson(cfg.benign_rate_per_s * cfg.duration_s))
```

```
        for t in np.sort(rng.uniform(0, cfg.duration_s - 1, size=n_flows)):
            kind = rng.choice(["dns", "http", "https"], p=[0.45, 0.25, 0.30])
            if kind == "dns":
                b.dns_exchange(c, float(t))
            else:
                b.tcp_session(c, str(rng.choice(WEB_SERVERS)), 80 if kind == "http" else 443,
                              float(t), int(rng.integers(60, 250)), int(rng.integers(150, 700)))

    # ---- anomalies ---------------------------------------------------------
    e = Ether(src=MAC_A, dst=MAC_B)
    for s in scenarios:
        t0 = s.start_window * WINDOW_SECONDS
        span = s.n_windows * WINDOW_SECONDS
        if s.kind == "port_scan":
            ports = rng.choice(np.arange(1, 10000), size=s.detail["ports"], replace=False)
            times = np.sort(rng.uniform(t0 + 0.2, t0 + span - 0.2, size=len(ports)))
            for p, t in zip(ports, times):
                b.add(e / b._ip(s.src_ip, s.detail["target"]) /
                      TCP(sport=int(rng.integers(40000, 60000)), dport=int(p), flags="S",
                          seq=int(rng.integers(1, 2**31))), float(t))
        elif s.kind == "traffic_spike":
            n = s.detail["packets_per_window"] * s.n_windows
            dst = str(rng.choice(WEB_SERVERS))
            for t in np.sort(rng.uniform(t0 + 0.2, t0 + span - 0.2, size=n)):
                b.add(e / b._ip(s.src_ip, dst) / UDP(sport=50505, dport=9999) /
                      Raw(_payload(rng, s.detail["payload"])), float(t))
        elif s.kind == "unknown_protocol":
            for t in np.sort(rng.uniform(t0 + 0.2, t0 + span - 0.2, size=s.detail["packets"])):
                proto = int(rng.choice([253, 254]))
                b.add(e / b._ip(s.src_ip, INTERNAL_SERVER, proto=proto) /
                      Raw(_payload(rng, int(rng.integers(40, 120)))), float(t))

    b.packets.sort(key=lambda p: float(p.time))
    pcap_path, labels_path = Path(pcap_path), Path(labels_path)
    pcap_path.parent.mkdir(parents=True, exist_ok=True)
    wrpcap(str(pcap_path), b.packets)

    # ---- labels: every (window, src) that appears gets a row ---------------
    anomalous = {}
    for s in scenarios:
        for wi in range(s.start_window, s.start_window + s.n_windows):
            anomalous[(wi * WINDOW_SECONDS, s.src_ip)] = s.kind
    seen = set()
    for p in b.packets:
        wi = int((float(p.time) - BASE_EPOCH) // WINDOW_SECONDS) * WINDOW_SECONDS
        seen.add((wi, p[IP].src))
    rows = sorted(seen)
    with open(labels_path, "w", newline="") as f:
        wr = csv.writer(f)
        wr.writerow(["window_start", "src_ip", "label"])
        for wi, src in rows:
            wr.writerow([wi, src, anomalous.get((wi, src), "benign")])

    digest = hashlib.sha256(pcap_path.read_bytes()).hexdigest()
    manifest = {
        "config": asdict(cfg),
        "packets": len(b.packets),
        "windows": n_windows,
        "window_seconds": WINDOW_SECONDS,
        "labelled_rows": len(rows),
        "anomalous_rows": sum(1 for k in rows if k in anomalous),
        "scenarios": [asdict(s) for s in scenarios],
        "pcap_sha256": digest,
        "pcap_bytes": pcap_path.stat().st_size,
    }
```

04-pcap-anomaly-dashboard/src/generate.py (continued)

```
    pcap_path.with_suffix(".manifest.json").write_text(json.dumps(manifest, indent=2))
    return manifest

```

```
"""Stream a PCAP file into a tidy per-packet pandas DataFrame.

Scapy's PcapReader reads one packet at a time, so memory stays flat even for
larger captures. Only IPv4 packets are kept (non-IP frames are counted and
skipped). Every field the detector needs is extracted here once.
"""
from __future__ import annotations

import logging
from pathlib import Path

import pandas as pd
from scapy.all import IP, TCP, UDP, ICMP, PcapReader  # type: ignore

logging.getLogger("scapy.runtime").setLevel(logging.ERROR)

# IANA protocol numbers the analyser "knows". Anything else is reported as an
# unknown protocol (configurable in config/detector.json).
PROTO_NAMES = {1: "ICMP", 6: "TCP", 17: "UDP"}

COLUMNS = ["ts", "src_ip", "dst_ip", "proto", "proto_name", "sport", "dport",
           "length", "syn", "ack"]


def parse_pcap(path: str | Path, max_packets: int | None = None) -> tuple[pd.DataFrame, dict]:
    """Return (packets_df, stats). ``max_packets`` bounds work on huge files."""
    rows = []
    stats = {"frames": 0, "ipv4": 0, "non_ip": 0}
    with PcapReader(str(path)) as reader:
        for pkt in reader:
            stats["frames"] += 1
            if max_packets is not None and stats["frames"] > max_packets:
                stats["frames"] -= 1
                stats["truncated"] = True
                break
            if IP not in pkt:
                stats["non_ip"] += 1
                continue
            ip = pkt[IP]
            proto = int(ip.proto)
            sport = dport = -1
            syn = ack = False
            if TCP in pkt:
                t = pkt[TCP]
                sport, dport = int(t.sport), int(t.dport)
                flags = int(t.flags)
                syn, ack = bool(flags & 0x02), bool(flags & 0x10)
            elif UDP in pkt:
                u = pkt[UDP]
                sport, dport = int(u.sport), int(u.dport)
            rows.append((float(pkt.time), ip.src, ip.dst, proto,
                         PROTO_NAMES.get(proto, f"IP-{proto}"), sport, dport,
                         len(pkt), syn, ack))
            stats["ipv4"] += 1
    df = pd.DataFrame(rows, columns=COLUMNS)
    return df, stats

```

```python
"""End-to-end analysis: PCAP -> packets -> windows -> detections -> metrics."""
from __future__ import annotations

import json
import time
from pathlib import Path

import pandas as pd

from .parser import parse_pcap
from .features import window_features, window_totals
from .detect import detect, load_config
from .evaluate import evaluate, join_labels


def analyze(pcap: str | Path, out_dir: str | Path, labels: str | Path | None = None,
            config: str | Path | None = None) -> dict:
    cfg = load_config(config)
    out = Path(out_dir); out.mkdir(parents=True, exist_ok=True)
    t = time.perf_counter()
    pkts, pstats = parse_pcap(pcap)
    t_parse = time.perf_counter() - t
    win = cfg["window_seconds"]
    feats = window_features(pkts, win, tuple(cfg["known_protocols"]))
    det = detect(feats, cfg)
    totals = window_totals(pkts, win)
    t_total = time.perf_counter() - t

    if labels:
        det = join_labels(det, pd.read_csv(labels))
    det.to_csv(out / "windows.csv", index=False)
    totals.to_csv(out / "window_totals.csv", index=False)
    alerts = det[det["combined_flag"]].sort_values(["window_start", "src_ip"])
    alerts.to_csv(out / "alerts.csv", index=False)

    summary = {
        "pcap": Path(pcap).name,
        "packets_parsed": pstats["ipv4"], "frames": pstats["frames"], "non_ip_frames": pstats["non_ip"],
        "window_seconds": win, "windows": int(det["window_start"].nunique()),
        "window_rows": int(len(det)), "unique_src_ips": int(det["src_ip"].nunique()),
        "protocols": sorted(pkts["proto_name"].unique().tolist()),
        "rule_alerts": int(det["rule_flag"].sum()), "if_alerts": int(det["if_flag"].sum()),
        "combined_alerts": int(det["combined_flag"].sum()),
        "rule_alerts_by_type": det.loc[det["rule_flag"], "rule_reason"].value_counts().to_dict(),
        "parse_seconds": round(t_parse, 3), "total_seconds": round(t_total, 3),
        "packets_per_second": round(pstats["ipv4"] / t_parse, 1) if t_parse else None,
    }
    if labels:
        summary["evaluation"] = evaluate(det.drop(columns=["label", "is_anomaly"]), pd.read_csv(labels))
    (out / "metrics.json").write_text(json.dumps(summary, indent=2))
    return summary


def benchmark(seeds, work_dir: str | Path, config: str | Path | None = None,
              difficulty: str = "standard") -> dict:
    """Generate one capture per seed and score it — a held-out check.

    Detector thresholds were chosen while developing against seed 42, so the
    seed-42 numbers are optimistic. Other seeds move every attack to different
    hosts/windows/intensities; summing their confusion counts gives a less
    biased (still synthetic!) estimate.
    """
    from .generate import GenConfig, generate
```

```
    from .evaluate import confusion
    cfg = load_config(config)
    work = Path(work_dir); work.mkdir(parents=True, exist_ok=True)
    frames = []
    for seed in seeds:
        pcap, lab = work / f"seed_{seed}.pcap", work / f"seed_{seed}_labels.csv"
        generate(GenConfig(seed=seed, difficulty=difficulty), pcap, lab)
        pkts, _ = parse_pcap(pcap)
        det = detect(window_features(pkts, cfg["window_seconds"], tuple(cfg["known_protocols"])),
cfg)
        m = join_labels(det, pd.read_csv(lab)); m["seed"] = seed
        frames.append(m)
        pcap.unlink(); lab.unlink(); pcap.with_suffix(".manifest.json").unlink()  # keep the work
dir small
    allm = pd.concat(frames, ignore_index=True)
    res = {"difficulty": difficulty, "seeds": list(seeds), "rows": int(len(allm)), "anomalous_rows
": int(allm["is_anomaly"].sum())}
    for name, col in [("rules", "rule_flag"), ("isolation_forest", "if_flag"), ("combined", "combi
ned_flag")]:
        res[name] = confusion(allm["is_anomaly"], allm[col])
    res["per_class"] = {
        cls: {"rows": int(len(g)), "rules_detected": int(g["rule_flag"].sum()),
              "rules_correct_type": int((g["rule_reason"] == cls).sum()), "if_detected": int(g["if
_flag"].sum())}
        for cls, g in allm[allm["is_anomaly"]].groupby("label")}
    return res

```

```
"""Charts (PNG) and a self-contained HTML report from analysis outputs."""
from __future__ import annotations

import base64
import html
import json
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
import pandas as pd  # noqa: E402

COLORS = {"benign": "#9aa5b1", "port_scan": "#d64545", "traffic_spike": "#e8833a",
          "unknown_protocol": "#7b4fd6"}


def chart_timeline(totals: pd.DataFrame, windows: pd.DataFrame, path: Path) -> None:
    piv = totals.pivot_table(index="window_start", columns="proto_name", values="packets",
                             aggfunc="sum", fill_value=0)
    fig, ax = plt.subplots(figsize=(11, 4.2))
    bottom = None
    for col in piv.columns:
        ax.bar(piv.index, piv[col], width=8, bottom=bottom, label=col)
        bottom = piv[col] if bottom is None else bottom + piv[col]
    alerts = windows[windows["rule_flag"]]
    ymax = float(bottom.max()) if bottom is not None else 1
    for _, r in alerts.iterrows():
        ax.axvline(r["window_start"], color=COLORS.get(r["rule_reason"], "k"), ls="--", lw=1, alph
a=.8)
        ax.text(r["window_start"], ymax * 1.02, r["rule_reason"].replace("_", " "), rotation=90,
                fontsize=7, va="bottom", ha="center", color=COLORS.get(r["rule_reason"], "k"))
    ax.set_ylim(0, ymax * 1.5)
    ax.set_xlabel("Window start (seconds from capture start, 10 s windows)")
    ax.set_ylabel("Packets")
    ax.set_title("Traffic per 10-second window by protocol, with rule alerts")
    ax.legend(fontsize=8, ncol=5, loc="upper left")
    fig.tight_layout(); fig.savefig(path, dpi=120); plt.close(fig)


def chart_scatter(windows: pd.DataFrame, path: Path) -> None:
    fig, ax = plt.subplots(figsize=(7.5, 5))
    lab = windows["label"] if "label" in windows else pd.Series("benign", index=windows.index)
    for cls in ["benign", "traffic_spike", "unknown_protocol", "port_scan"]:
        sub = windows[lab == cls]
        if len(sub):
            ax.scatter(sub["packets"], sub["unique_dst_ports"] + 1, s=18 if cls == "benign" else 6
0,
                       c=COLORS[cls], label=f"{cls} (n={len(sub)})", alpha=.6 if cls == "benign" e
lse .95,
                       edgecolors="k" if cls != "benign" else "none", linewidths=.5)
    fl = windows[windows["if_flag"]]
    ax.scatter(fl["packets"], fl["unique_dst_ports"] + 1, s=140, facecolors="none",
               edgecolors="#111", linewidths=1, label=f"Isolation Forest flag (n={len(fl)})")
    ax.set_xscale("log"); ax.set_yscale("log")
    ax.set_xlabel("Packets in window (log)"); ax.set_ylabel("Distinct destination ports + 1 (log)"
)
    ax.set_title("Each dot = one source IP in one 10 s window (colour = ground truth)")
    ax.legend(fontsize=7.5)
    fig.tight_layout(); fig.savefig(path, dpi=120); plt.close(fig)


def chart_scores(windows: pd.DataFrame, path: Path) -> None:
    fig, ax = plt.subplots(figsize=(7.5, 4))
    lab = windows.get("label", pd.Series("benign", index=windows.index))
```

```
    ben = windows.loc[lab == "benign", "if_score"]; ano = windows.loc[lab != "benign", "if_score"]
    ax.hist(ben, bins=40, color=COLORS["benign"], alpha=.8, label=f"benign rows (n={len(ben)})")
    if len(ano):
        ax.hist(ano, bins=20, color=COLORS["port_scan"], alpha=.8, label=f"anomalous rows (n={len(ano)})")
    if windows["if_flag"].any():
        thr = windows.loc[windows["if_flag"], "if_score"].min()
        ax.axvline(thr, color="k", ls="--", lw=1, label=f"flag cut-off ({thr:.3f})")
    ax.set_yscale("log"); ax.set_xlabel("Isolation Forest anomaly score (higher = more unusual)")
    ax.set_ylabel("Rows (log)"); ax.set_title("Isolation Forest score distribution")
    ax.legend(fontsize=8)
    fig.tight_layout(); fig.savefig(path, dpi=120); plt.close(fig)


def chart_benchmark(bench: dict, path: Path) -> None:
    """Grouped bars of precision/recall per detector for each benchmark."""
    rows = []
    for name, b in bench.items():
        for det in ["rules", "isolation_forest", "combined"]:
            rows.append((name, det, b[det]["precision"], b[det]["recall"]))
    df = pd.DataFrame(rows, columns=["bench", "detector", "precision", "recall"])
    fig, axes = plt.subplots(1, len(bench), figsize=(5.2 * len(bench), 4), sharey=True, squeeze=False)
    for ax, (name, sub) in zip(axes[0], df.groupby("bench", sort=False)):
        x = range(len(sub))
        ax.bar([i - .2 for i in x], sub["precision"], width=.4, label="precision", color="#3b6fb6")
        ax.bar([i + .2 for i in x], sub["recall"], width=.4, label="recall", color="#5aa469")
        for i, (p, r) in enumerate(zip(sub["precision"], sub["recall"])):
            ax.text(i - .2, p + .02, f"{p:.2f}", ha="center", fontsize=8)
            ax.text(i + .2, r + .02, f"{r:.2f}", ha="center", fontsize=8)
        ax.set_xticks(list(x)); ax.set_xticklabels(["Rules", "Isolation\nForest", "Rules OR\nForest"])
        b = bench[name]
        ax.set_title(f"{name}\n{b['anomalous_rows']} anomalous of {b['rows']} rows", fontsize=10)
        ax.set_ylim(0, 1.15)
    axes[0][0].set_ylabel("Score"); axes[0][0].legend(fontsize=8, loc="lower left")
    fig.suptitle("Held-out synthetic seeds: precision / recall per detector", fontsize=11)
    fig.tight_layout(); fig.savefig(path, dpi=120); plt.close(fig)


def build_report(analysis_dir: str | Path, out_html: str | Path, charts_dir: str | Path,
                 bench: dict | None = None) -> list[Path]:
    a = Path(analysis_dir); charts = Path(charts_dir); charts.mkdir(parents=True, exist_ok=True)
    windows = pd.read_csv(a / "windows.csv"); totals = pd.read_csv(a / "window_totals.csv")
    metrics = json.loads((a / "metrics.json").read_text())
    made = [charts / "traffic_timeline.png", charts / "feature_scatter.png", charts / "iforest_scores.png"]
    chart_timeline(totals, windows, made[0]); chart_scatter(windows, made[1]); chart_scores(windows, made[2])
    if bench:
        made.append(charts / "benchmark_precision_recall.png"); chart_benchmark(bench, made[-1])

    def img(p: Path) -> str:
        return f'<img src="{base64.b64encode(p.read_bytes()).decode()}" style="max-width:100%">'

    alerts = windows[windows["combined_flag"]]
    cols = [c for c in ["window_start", "src_ip", "packets", "bytes", "unique_dst_ports", "unknown_proto_packets",
                        "rule_reason", "if_score", "label"] if c in alerts]
    ev = metrics.get("evaluation")
    ev_html = ""
    if ev:
        trs = "".join(f"<tr><td>{n}</td><td>{ev[k]['precision_str']} = {ev[k]['precision']:.3f}</t
```

```
d>"
                    f"<td>{ev[k]['recall_str']} = {ev[k]['recall']:.3f}</td><td>{ev[k]['fp']}</td></tr>"
                    for n, k in [("Rules", "rules"), ("Isolation Forest", "isolation_forest"), ("Rules OR Forest", "combined")])
        ev_html = (f"<h2>Evaluation vs generator labels ({ev['anomalous_rows']} anomalous of {ev['rows']} rows)</h2>"
                   f"<table><tr><th>Detector</th><th>Precision</th><th>Recall</th><th>False positives</th></tr>{trs}</table>")
    doc = f"""<!doctype html><html><head><meta charset="utf-8"><title>PCAP anomaly report</title>
<style>body{{font-family:system-ui,sans-serif;max-width:1000px;margin:auto;padding:16px}}
table{{border-collapse:collapse;font-size:13px}}td,th{{border:1px solid #ccc;padding:4px 8px}}
.note{{background:#fff6dd;padding:8px;border-left:4px solid #e0a800}}</style></head><body>
<h1>PCAP anomaly report: {html.escape(metrics['pcap'])}</h1>
<p class="note">Synthetic, generated traffic. Results show the detector works on its own labelled test data; they are not a production validation.</p>
<p>{metrics['packets_parsed']} packets &middot; {metrics['windows']} windows of {metrics['window_seconds']} s &middot;
{metrics['window_rows']} (window, source) rows &middot; protocols: {html.escape(', '.join(metrics['protocols']))} &middot;
analysed in {metrics['total_seconds']} s</p>
{ev_html}
<h2>Alerts ({len(alerts)})</h2>{alerts[cols].to_html(index=False)}
<h2>Charts</h2>{''.join(img(p) for p in made)}
</body></html>"""
    Path(out_html).write_text(doc)
    return made

```

04-pcap-anomaly-dashboard/tests/conftest.py

```python
import logging
import pytest

from src.generate import GenConfig, generate

logging.getLogger("scapy.runtime").setLevel(logging.ERROR)


@pytest.fixture(scope="session")
def sample(tmp_path_factory):
    """One deterministic capture shared by the slower end-to-end tests."""
    d = tmp_path_factory.mktemp("cap")
    pcap, labels = d / "s.pcap", d / "s_labels.csv"
    manifest = generate(GenConfig(seed=42), pcap, labels)
    return pcap, labels, manifest

```

04-pcap-anomaly-dashboard/tests/helpers.py

```
"""Tiny hand-built captures with exactly known contents."""
from scapy.all import IP, TCP, UDP, Ether, Raw, wrpcap  # type: ignore

T0 = 1_700_000_000.0


def pkt(t, src, dst, layer, payload=b""):
    p = Ether() / IP(src=src, dst=dst) / layer
    if payload:
        p = p / Raw(payload)
    p.time = T0 + t
    return p


def write(path, packets):
    wrpcap(str(path), packets)
    return path

```

04-pcap-anomaly-dashboard/tests/test_cli_app.py

```
import json
import subprocess
import sys
from pathlib import Path

import pytest

ROOT = Path(__file__).resolve().parent.parent


def run(*args, cwd=ROOT):
    return subprocess.run([sys.executable, "cli.py", *args], cwd=cwd, capture_output=True, text=True, timeout=240)


def test_cli_help():
    r = run("--help")
    assert r.returncode == 0 and "generate" in r.stdout and "analyze" in r.stdout and "report" in r.stdout


def test_cli_generate_analyze_report(tmp_path):
    pcap = tmp_path / "c.pcap"
    assert run("generate", "--seed", "5", "--duration", "200", "--out", str(pcap)).returncode == 0
    labels = tmp_path / "c_labels.csv"
    r = run("analyze", "--pcap", str(pcap), "--labels", str(labels), "--out", str(tmp_path / "a"))
    assert r.returncode == 0, r.stderr
    m = json.loads((tmp_path / "a" / "metrics.json").read_text())
    assert m["evaluation"]["rows"] == m["window_rows"]
    r = run("report", "--analysis", str(tmp_path / "a"), "--html", str(tmp_path / "r.html"),
            "--charts", str(tmp_path / "ch"))
    assert r.returncode == 0, r.stderr
    assert (tmp_path / "r.html").read_text().count("data:image/png;base64") == 3


def test_benchmark_small(tmp_path):
    from src.pipeline import benchmark
    b = benchmark([11, 12], tmp_path / "w", difficulty="hard")
    assert b["rows"] > 0 and b["anomalous_rows"] > 0
    assert b["rules"]["tp"] + b["rules"]["fn"] == b["anomalous_rows"]
    assert list((tmp_path / "w").iterdir()) == []      # temp captures cleaned up


def test_streamlit_app_runs():
    from streamlit.testing.v1 import AppTest
    at = AppTest.from_file(str(ROOT / "app.py"), default_timeout=120).run()
    assert not at.exception
    assert [t.label for t in at.tabs] == ["Traffic timeline", "Anomaly table", "Feature space", "Evaluation"]

```

04-pcap-anomaly-dashboard/tests/test_detect_eval.py

```
import json

import numpy as np
import pandas as pd
import pytest

from src.detect import detect, load_config, robust_z, rule_detect
from src.evaluate import confusion, evaluate
from src.features import FEATURES


def frame(rows):
    base = {f: 0 for f in FEATURES}
    out = []
    for i, r in enumerate(rows):
        d = dict(base, window_start=10 * (i % 30), src_ip=f"10.0.0.{i % 200}")
        d.update(r)
        out.append(d)
    return pd.DataFrame(out)


def benign_rows(n=200, seed=0):
    rng = np.random.default_rng(seed)
    rows = []
    for _ in range(n):
        p = int(rng.integers(4, 20))
        rows.append(dict(packets=p, bytes=p * int(rng.integers(80, 300)), unique_dst_ips=int(rng.integers(1, 3)),
                         unique_dst_ports=int(rng.integers(1, 4)), syn_packets=1, syn_ratio=round(1 / p, 3),
                         mean_pkt_len=150.0, tcp_packets=p // 2, udp_packets=p - p // 2))
    return rows


CFG = load_config()


def test_robust_z_ignores_outlier():
    z = robust_z(pd.Series([10, 11, 9, 10, 12, 10, 1000]))
    assert z.iloc[-1] > 100 and abs(z.iloc[0]) < 2


def test_port_scan_flag():
    df = frame(benign_rows() + [dict(packets=400, bytes=400 * 54, unique_dst_ips=1, unique_dst_ports=400,
                                     syn_packets=400, syn_ratio=1.0, mean_pkt_len=54, tcp_packets=400)])
    out = rule_detect(df, CFG)
    assert out.iloc[-1]["rule_reason"] == "port_scan"
    assert not out.iloc[:-1]["rule_flag"].any()


def test_many_ports_without_syns_is_not_a_scan():
    # e.g. a busy server answering many client ephemeral ports (SYN-ACK / data, not bare SYNs)
    df = frame(benign_rows() + [dict(packets=12, bytes=1200, unique_dst_ips=8, unique_dst_ports=80,
                                     syn_packets=0, syn_ratio=0.0, mean_pkt_len=100)])
    assert rule_detect(df, CFG).iloc[-1]["rule_reason"] != "port_scan"


def test_traffic_spike_flag():
    df = frame(benign_rows() + [dict(packets=300, bytes=300 * 300, unique_dst_ips=1, unique_dst_ports=1,
                                     mean_pkt_len=300, udp_packets=300)])
    out = rule_detect(df, CFG)
```

```
    assert out.iloc[-1]["rule_reason"] == "traffic_spike"
    assert out.iloc[-1]["z_bytes"] >= CFG["traffic_spike"]["robust_z_threshold"]


def test_unknown_protocol_flag_and_config_threshold():
    df = frame(benign_rows() + [dict(packets=3, bytes=300, unique_dst_ips=1, unknown_proto_packets=3)])
    assert rule_detect(df, CFG).iloc[-1]["rule_reason"] == "unknown_protocol"
    cfg = json.loads(json.dumps(CFG)); cfg["unknown_protocol"]["min_packets"] = 5
    assert rule_detect(df, cfg).iloc[-1]["rule_reason"] == ""


def test_benign_only_has_no_rule_alerts():
    assert not rule_detect(frame(benign_rows(500, seed=3)), CFG)["rule_flag"].any()


def test_isolation_forest_deterministic_and_finds_outlier():
    df = frame(benign_rows() + [dict(packets=400, bytes=400 * 54, unique_dst_ports=400, syn_packets=400,
                                     syn_ratio=1.0, mean_pkt_len=54, unique_dst_ips=1)])
    a, b = detect(df, CFG), detect(df, CFG)
    assert a["if_score"].equals(b["if_score"])
    assert a.iloc[-1]["if_flag"] and a["if_score"].idxmax() == len(df) - 1
    assert a["combined_flag"].equals(a["rule_flag"] | a["if_flag"])


def test_confusion_counts_and_zero_division():
    c = confusion([1, 1, 0, 0, 0], [1, 0, 1, 0, 0])
    assert (c["tp"], c["fp"], c["fn"], c["tn"]) == (1, 1, 1, 2)
    assert c["precision"] == 0.5 and c["recall"] == 0.5 and c["precision_str"] == "1/2"
    z = confusion([0, 0], [0, 0])
    assert z["precision"] == 0.0 and z["recall"] == 0.0


def test_end_to_end_on_generated_capture(sample):
    from src.parser import parse_pcap
    from src.features import window_features
    pcap, labels, m = sample
    pk, _ = parse_pcap(pcap)
    det = detect(window_features(pk, 10), CFG)
    ev = evaluate(det, pd.read_csv(labels))
    assert ev["anomalous_rows"] == m["anomalous_rows"]
    # Every scenario type is caught by the matching rule on the development seed.
    for cls, c in ev["per_class"].items():
        assert c["rules_correct_type"] == c["rows"], cls
    assert ev["rules"]["fp"] == 0
    assert ev["isolation_forest"]["false_positive_rate"] < 0.05

```

```python
import hashlib

import pandas as pd
import pytest

from src.generate import GenConfig, generate


def sha(p):
    return hashlib.sha256(p.read_bytes()).hexdigest()


def test_same_seed_is_byte_identical(tmp_path):
    m1 = generate(GenConfig(seed=7, duration_s=120), tmp_path / "a.pcap", tmp_path / "a.csv")
    m2 = generate(GenConfig(seed=7, duration_s=120), tmp_path / "b.pcap", tmp_path / "b.csv")
    assert sha(tmp_path / "a.pcap") == sha(tmp_path / "b.pcap") == m1["pcap_sha256"] == m2["pcap_sha256"]
    assert (tmp_path / "a.csv").read_text() == (tmp_path / "b.csv").read_text()


def test_different_seed_differs(tmp_path):
    generate(GenConfig(seed=1, duration_s=120), tmp_path / "a.pcap", tmp_path / "a.csv")
    generate(GenConfig(seed=2, duration_s=120), tmp_path / "b.pcap", tmp_path / "b.csv")
    assert sha(tmp_path / "a.pcap") != sha(tmp_path / "b.pcap")


def test_labels_cover_every_scenario(sample):
    _, labels, m = sample
    df = pd.read_csv(labels)
    assert set(df["label"]) == {"benign", "port_scan", "traffic_spike", "unknown_protocol"}
    expected = sum(s["n_windows"] for s in m["scenarios"])
    assert (df["label"] != "benign").sum() == expected == m["anomalous_rows"]
    assert not df.duplicated(["window_start", "src_ip"]).any()


def test_only_documentation_or_private_addresses(sample):
    _, labels, _ = sample
    ips = pd.read_csv(labels)["src_ip"]
    assert ips.str.match(r"^(10\.0\.0\.|198\.51\.100\.|203\.0\.113\.)").all()


def test_hard_mode_and_bad_difficulty(tmp_path):
    m = generate(GenConfig(seed=3, difficulty="hard"), tmp_path / "h.pcap", tmp_path / "h.csv")
    assert all(s["detail"].get("ports", 0) < 150 for s in m["scenarios"] if s["kind"] == "port_scan")
    with pytest.raises(ValueError):
        generate(GenConfig(seed=3, difficulty="extreme"), tmp_path / "x.pcap", tmp_path / "x.csv")


def test_benign_dns_is_real_dns(sample):
    from scapy.all import PcapReader  # type: ignore
    from scapy.layers.dns import DNS  # type: ignore
    pcap, _, _ = sample
    names = set()
    with PcapReader(str(pcap)) as r:
        for i, p in enumerate(r):
            if DNS in p and p[DNS].qd is not None:
                names.add(p[DNS].qd.qname.decode().rstrip("."))
            if i > 3000:
                break
    assert names and all(n.endswith((".com", ".org", ".net")) and ".example." in "." + n for n in names)

```

```python
from scapy.all import TCP, UDP  # type: ignore
from scapy.layers.inet import IP  # type: ignore

from src.parser import parse_pcap
from src.features import window_features
from tests.helpers import pkt, write


def test_parser_fields(tmp_path):
    p = write(tmp_path / "t.pcap", [
        pkt(0.5, "10.0.0.1", "10.0.0.2", TCP(sport=1234, dport=80, flags="S")),
        pkt(1.0, "10.0.0.2", "10.0.0.1", TCP(sport=80, dport=1234, flags="SA")),
        pkt(2.0, "10.0.0.1", "10.0.0.9", UDP(sport=5000, dport=53), b"x" * 10),
    ])
    df, stats = parse_pcap(p)
    assert stats == {"frames": 3, "ipv4": 3, "non_ip": 0}
    assert df["proto_name"].tolist() == ["TCP", "TCP", "UDP"]
    assert df["syn"].tolist() == [True, True, False]
    assert df["ack"].tolist() == [False, True, False]
    assert df.loc[2, "dport"] == 53


def test_parser_unknown_protocol_and_non_ip(tmp_path):
    from scapy.all import Ether, ARP  # type: ignore
    arp = Ether() / ARP(); arp.time = 1_700_000_000.0
    raw = pkt(0.1, "10.0.0.1", "10.0.0.2", b"")
    raw[IP].proto = 253
    p = write(tmp_path / "u.pcap", [arp, raw])
    df, stats = parse_pcap(p)
    assert stats["non_ip"] == 1 and stats["ipv4"] == 1
    assert df.loc[0, "proto"] == 253 and df.loc[0, "proto_name"] == "IP-253"
    assert df.loc[0, "sport"] == -1


def test_parser_max_packets(tmp_path):
    p = write(tmp_path / "m.pcap", [pkt(i, "10.0.0.1", "10.0.0.2", UDP(dport=53)) for i in range(5
)])
    df, stats = parse_pcap(p, max_packets=3)
    assert len(df) == 3 and stats.get("truncated")


def test_window_counts_exact(tmp_path):
    # Windows are aligned to multiples of 10 s of epoch time: 0-9.99 / 10-19.99 / 20-29.99
    pk = [pkt(t, "10.0.0.1", "10.0.0.2", UDP(sport=1, dport=53), b"a" * 20) for t in (0.1, 3.0, 9.
99)]
    pk += [pkt(10.0, "10.0.0.1", "10.0.0.2", UDP(sport=1, dport=53))]
    pk += [pkt(25.0, "10.0.0.3", "10.0.0.2", TCP(dport=d, flags="S")) for d in (22, 80, 443)]
    df, _ = parse_pcap(write(tmp_path / "w.pcap", pk))
    f = window_features(df, 10).set_index(["window_start", "src_ip"])
    assert list(f.index) == [(0, "10.0.0.1"), (10, "10.0.0.1"), (20, "10.0.0.3")]
    assert f.loc[(0, "10.0.0.1"), "packets"] == 3
    assert f.loc[(10, "10.0.0.1"), "packets"] == 1
    assert f.loc[(0, "10.0.0.1"), "bytes"] == 3 * (14 + 20 + 8 + 20)
    r = f.loc[(20, "10.0.0.3")]
    assert r["unique_dst_ports"] == 3 and r["syn_packets"] == 3 and r["syn_ratio"] == 1.0


def test_window_alignment_independent_of_first_packet(tmp_path):
    # Capture starting at 7 s: the packet at 12 s must still land in window 10.
    pk = [pkt(7.0, "10.0.0.1", "10.0.0.2", UDP(dport=53)), pkt(12.0, "10.0.0.1", "10.0.0.2", UDP(d
port=53))]
    df, _ = parse_pcap(write(tmp_path / "a.pcap", pk))
    assert window_features(df, 10)["window_start"].tolist() == [0, 10]


```

```
63 def test_parsed_count_matches_generator(sample):
64     pcap, _, m = sample
65     df, stats = parse_pcap(pcap)
66     assert stats["ipv4"] == len(df) == m["packets"]
67     f = window_features(df, 10)
68     assert f["packets"].sum() == m["packets"]
69     assert f["window_start"].nunique() == m["windows"]
70     assert len(f) == m["labelled_rows"]
71
```

04-pcap-anomaly-dashboard/.gitignore

```
__pycache__/
*.pyc
.pytest_cache/
.venv/
venv/
*.db
data/bench_tmp*/
outputs/analysis/window_totals.csv.bak
.streamlit/secrets.toml

```

04-pcap-anomaly-dashboard/config/detector.json

```
{
  "window_seconds": 10,
  "known_protocols": [1, 6, 17],
  "port_scan": {"min_unique_dst_ports": 50, "min_syn_ratio": 0.6},
  "traffic_spike": {"robust_z_threshold": 3.5, "min_packets": 50},
  "unknown_protocol": {"min_packets": 1},
  "isolation_forest": {"n_estimators": 200, "contamination": 0.02, "random_state": 0}
}

```

```
{
  "config": {
    "seed": 42,
    "duration_s": 600,
    "n_clients": 14,
    "benign_rate_per_s": 0.18,
    "n_port_scans": 3,
    "n_spikes": 3,
    "n_unknown": 2,
    "difficulty": "standard"
  },
  "packets": 8972,
  "windows": 60,
  "window_seconds": 10,
  "labelled_rows": 996,
  "anomalous_rows": 10,
  "scenarios": [
    {
      "kind": "port_scan",
      "src_ip": "203.0.113.66",
      "start_window": 30,
      "n_windows": 1,
      "detail": {
        "target": "10.0.0.50",
        "ports": 436
      }
    },
    {
      "kind": "port_scan",
      "src_ip": "203.0.113.67",
      "start_window": 53,
      "n_windows": 1,
      "detail": {
        "target": "10.0.0.50",
        "ports": 425
      }
    },
    {
      "kind": "port_scan",
      "src_ip": "203.0.113.68",
      "start_window": 20,
      "n_windows": 1,
      "detail": {
        "target": "10.0.0.50",
        "ports": 422
      }
    },
    {
      "kind": "traffic_spike",
      "src_ip": "10.0.0.20",
      "start_window": 26,
      "n_windows": 2,
      "detail": {
        "packets_per_window": 338,
        "payload": 273
      }
    },
    {
      "kind": "traffic_spike",
      "src_ip": "10.0.0.11",
      "start_window": 9,
      "n_windows": 1,
      "detail": {
        "packets_per_window": 317,
        "payload": 218
```

```
      }
    },
    {
      "kind": "traffic_spike",
      "src_ip": "10.0.0.14",
      "start_window": 48,
      "n_windows": 2,
      "detail": {
        "packets_per_window": 275,
        "payload": 244
      }
    },
    {
      "kind": "unknown_protocol",
      "src_ip": "10.0.0.17",
      "start_window": 57,
      "n_windows": 1,
      "detail": {
        "packets": 13
      }
    },
    {
      "kind": "unknown_protocol",
      "src_ip": "10.0.0.15",
      "start_window": 41,
      "n_windows": 1,
      "detail": {
        "packets": 13
      }
    }
  ],
  "pcap_sha256": "31d779d3363caf484c2b1cfa8c336ae9fa83dc092df4253fbabeaa0576a15385",
  "pcap_bytes": 1511276
}
```

04-pcap-anomaly-dashboard/pytest.ini

```
[pytest]
pythonpath = .
testpaths = tests
timeout = 300
filterwarnings =
    ignore::DeprecationWarning
markers =
    network: needs internet (none in this project)
    slow: end-to-end tests that generate and analyse full captures

```

04-pcap-anomaly-dashboard/scripts/run_demo.sh

```
#!/usr/bin/env bash
# Reproduce every artifact in outputs/ from scratch (about 2 minutes on a laptop CPU).
set -euo pipefail
cd "$(dirname "$0")/.."
PY=${PY:-python}
$PY cli.py generate --seed 42 --out data/sample.pcap
$PY cli.py analyze --pcap data/sample.pcap --labels data/sample_labels.csv --out outputs/analysis
$PY cli.py benchmark --seeds 1-20 --difficulty standard --work data/bench_tmp_std --out outputs/benchmark_standard.json
$PY cli.py benchmark --seeds 101-120 --difficulty hard --work data/bench_tmp_hard --out outputs/benchmark_hard.json
$PY cli.py report --analysis outputs/analysis --html outputs/report.html --charts outputs/charts \
    --bench outputs/benchmark_standard.json outputs/benchmark_hard.json

```